

UNIVERSIDAD DE VALLADOLID
GRADO EN INGENIERÍA BIOMÉDICA



PROCESADO DE SEÑAL E IMAGEN MÉDICA

Práctica 3.2. Redes neuronales

CURSO 2024/2025

ÍNDICE

1. OBJETIVOS Y METODOLOGÍA	3
2. CONSIDERACIONES PREVIAS	3
3. TEMPORIZACIÓN	4
4. TOOLBOX DE REDES NEURONALES	4
4.1. Aprendiendo la función XOR	4
4.2. Un problema real	7
6. EVALUACIÓN	9
7. BIBLIOGRAFÍA	9

1. OBJETIVOS Y METODOLOGÍA

En esta segunda parte de la Práctica 3 se pondrán en práctica algunos conceptos relacionados con la clasificación empleando redes neuronales. Por un lado, se tratará de que los alumnos conozcan algunas de las funciones relacionadas con las redes neuronales incluidas en las distribuciones de Matlab® *Neural Network Toolbox™*, *Statistics and Machine Learning Toolbox™* o *Deep Learning Toolbox™*, dependiendo de la versión de Matlab®. Asimismo, se guiará a los alumnos en la resolución de algunos problemas sencillos empleando estas funciones. Finalmente, se proponen varias actividades para que los alumnos implementen nuevas funcionalidades y analicen los resultados obtenidos. Los objetivos de la práctica consistirán en:

- (i) Conocer las funciones básicas de Matlab® para la creación, entrenamiento y testeo de redes neuronales tipo perceptrón multicapa.
- (ii) Aplicar las funciones básicas de Matlab® para realizar clasificación en problemas sencillos.
- (iii) Aplicar técnicas vistas sobre un problema médico con datos reales e interpretar los resultados obtenidos.

2. CONSIDERACIONES PREVIAS

Debido al limitado tiempo disponible en el laboratorio, es **IMPRESINDIBLE** plantearse la resolución de los ejercicios antes de la correspondiente sesión práctica y realizar el trabajo previo a cada sesión indicado en el guion de la práctica. De este modo, es posible sacar el máximo partido al tiempo pasado en el laboratorio.

Para realizar las simulaciones de cada apartado, se recomienda generar un archivo “.m” en el que se incluya el código de Matlab® que dé respuesta a las cuestiones planteadas. En cada uno de los archivos de simulación generados, incluya a modo de cabecera las siguientes líneas de código:

```
clear      % Se borran las variables del espacio de trabajo
clc        % Se borra la pantalla de la ventana de comandos
close all  % Se cierran todas las figuras abiertas
```

Existen varias herramientas y *toolboxes* para la implementación *software* de redes neuronales. Para esta práctica, emplearemos como base el *toolbox* de redes neuronales incluido en la distribución de Matlab. Se advierte a los alumnos que puede que algunas de las funciones presentadas a lo largo de la práctica no sean iguales en todas las versiones de Matlab®. Se recomienda a los alumnos que, en estos casos, consulten la ayuda de Matlab® correspondiente a su propia versión de este *software* para encontrar las funciones equivalentes a las presentadas aquí.

A la hora de programar las simulaciones indicadas, comente el código generado para que pueda reaprovecharlo fácilmente más adelante si fuera necesario. Asimismo, realice una programación modular, encapsulando en funciones independientes los *scripts* que realicen tareas muy concretas.

3. TEMPORIZACIÓN

Este segundo bloque de la práctica 3 tiene asociada una única sesión de laboratorio que tendrá lugar el día **14 de mayo para el grupo L2 (11:00-13:00) y 21 de mayo para el grupo L1 (11:00-13:00)**. La práctica se llevará a cabo en el laboratorio **Sim 3244** de la Escuela de Ingenierías Industriales (sede Doctor Mergelina).

4. REDES NEURONALES

En primer lugar, es necesario conocer las funcionalidades que ofrece Matlab® para la resolución de problemas empleando redes neuronales. Para ello, se insta a los alumnos a que consulten la ayuda de Matlab® (*Neural Network Toolbox™*, *Statistics and Machine Learning Toolbox™* o *Deep Learning Toolbox™*, dependiendo de la versión de Matlab®).

4.1. Aprendiendo la función XOR

Para comprender el funcionamiento de las redes neuronales y para comenzar a manejar las funciones del *toolbox*, vamos a utilizar un **perceptrón multicapa** para resolver la función XOR.

A	B	A xor B
1	1	0
1	0	1
0	1	1
0	0	0

Como tenemos **dos entradas** (A y B), necesitaremos **dos neuronas de entrada**. Asimismo, vamos a **distinguir entre dos estados** ('0' y '1'), con lo que necesitaremos **una neurona de salida**. En este caso, consideraremos que un patrón de entrada pertenece a la clase '1' si la salida de la red neuronal para dicha entrada está por encima de un determinado **umbral** (típicamente **0.5**). Asimismo, si la salida está por debajo del umbral, consideraremos que el patrón de entrada pertenece a la clase '0'.

A continuación vamos a crear una red neuronal de tipo **MLP** con **dos nodos en la capa oculta** empleando la función `feedforwardnet` (en versiones de Matlab® más antiguas, la función para crear una red MLP era `newff`).

```
>> net = feedforwardnet(2);
```

Explique cual es el significado del argumento de la expresión anterior. Para visualizar la estructura de la red creada se puede emplear el comando:

```
>> view(net);
```

Además, para ver todos los parámetros que se pueden configurar en la red se puede emplear el comando:

```
>> display(net);
```

Vamos a configurar la **función de activación** de las neuronas. Para ello, se va a elegir como función de activación la función **logsig**, que se muestra en la siguiente figura:

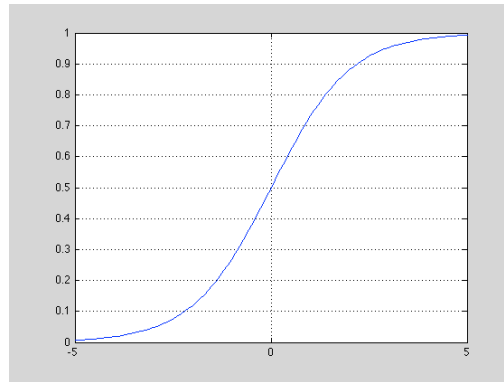


Figura 1. Función de activación logsig.

¿Por qué se ha elegido esta función? ¿Se podrían haber empleado otras funciones de activación? Acuda a la ayuda de Matlab® para ver las posibles funciones de activación para una red MLP.

También es recomendable ver si se pueden configurar parámetros como la dimensionalidad de las entradas y salidas, el rango de valores en el que se encuentran los vectores de entrada, el algoritmo de entrenamiento, los parámetros del algoritmo de entrenamiento, etc. Revise las opciones disponibles para tenerlas en cuenta en el desarrollo de la práctica.

Veamos cómo funciona la red sin entrenar. Para ello creamos una matriz con las entradas (serán las columnas de la matriz) con el nombre `input` y un vector con las salidas deseadas de la red (`target`), llamado también vector de objetivos o vector de valores objetivo:

```
>> input = [1 1 0 0; 1 0 1 0];
>> target = [0 1 1 0];
```

Una vez que se conoce como deben de ser las entradas y salidas, se puede utilizar el siguiente comando para configurar correctamente la red:

```
>> net = configure(net, input, target);
```

¿Qué parámetros de la red se ajustan con el comando anterior? ¿Se podría conseguir el mismo resultado variando a mano los valores de los parámetros que se visualizan cuando se emplea el comando `display(net)`?

Para ver el resultado que se obtiene para las entradas dadas por la matriz `input` con la red sin entrenar, simulamos la red con dichas entradas:

```
>> output = sim(net, input);
output=
    0.0289    0.1225    0.0311    0.7257
```

Vemos que la salida es muy diferente de lo que deberíamos haber obtenido (0 1 1 0). Esto es debido a que los pesos de la red se han inicializado aleatoriamente y la red no ha sido entrenada (de esta forma, cada vez que se ejecuten estos comandos, las salidas pueden ser distintas). Se recomienda a los alumnos que dibujen conjuntamente las salidas reales (`output`) y los objetivos que se pretenden conseguir (`target`).

A la vista de los resultados, está claro que con los **pesos** actuales de la red no se obtienen buenos resultados. Para ver el valor que tienen se puede acceder a los mismos utilizando los siguientes comandos:

```
>> net.IW{1,1}
ans =
    5.1071   -6.0529
    5.5479    5.6516
>> net.LW{2,1}
ans =
    1.7495   -5.3197
```

Podríamos cambiar cualquier peso de la red y volver a evaluar sus resultados, que variarán con respecto a los obtenidos anteriormente:

```
>> net.IW{1,1}(1,2)=5;
>> output = sim(net, input);
output =
    0.1434    0.1225    0.1185    0.7257
```

Así, ahora los pesos de la red quedarían del siguiente modo:

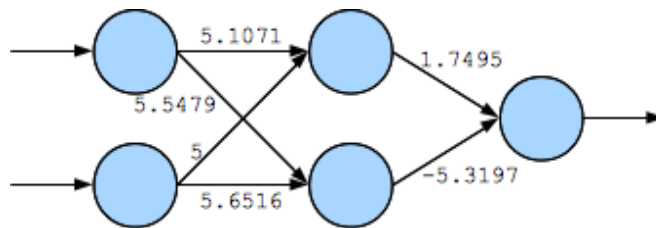


Figura 2. Representación de la red con los pesos finales.

Se recomienda a los alumnos que varíen algunos pesos más y comprueben las salidas. Pueden, al igual que antes, visualizar los resultados que se van obteniendo junto con los valores objetivo para ver de forma gráfica la precisión de la red.

Los **algoritmos de entrenamiento** permiten que los **pesos se calculen de forma automática** de acuerdo con ciertos criterios. Por lo tanto, es más adecuado entrenar la red que ir variando los pesos a mano hasta obtener un buen resultado. Para ello utilizamos la función **train**:

```
>> net = train(net, input, target);
```

Las diferentes versiones de Matlab permiten ver la evolución de los resultados durante el entrenamiento de forma gráfica (aunque hay diferencias entre las versiones). Si simulamos la red una vez entrenada, empleando los mismos vectores de entrada, obtendríamos ya algo muy parecido a los valores objetivo:

```
>> output = sim(net, input);
output=
    0.0015    0.9990    0.9990    0.0009
```

Visualice y anote cuales son los pesos de la red una vez entrenada. Investigue cuál es el algoritmo de entrenamiento empleado en este caso y qué otras opciones habría.

Hay ocasiones en los que, al simular la red entrenada, las salidas deseadas siguen sin corresponder con los valores objetivo deseados. Se puede tratar de **mejorar el entrenamiento variando los parámetros de la red**: número de nodos en la capa oculta, algoritmo de entrenamiento, valores de los parámetros del algoritmo de entrenamiento, etc. Se recomienda a los alumnos que prueben algunas de estas opciones para la función XOR.

Ejercicio optativo: Emplee las herramientas gráficas de Matlab® para repetir este ejercicio. Existen varias opciones para ello. Emplee el comando `nnstart` y seleccione la opción **Pattern Recognition** con este propósito. Puede visualizar también otras opciones disponibles empleando el comando `nnstart`. También puede probar la aplicación **Classification Learner**, disponible con las versiones más recientes de Matlab®.

4.2. Un problema real

En este último ejercicio vamos a emplear el fichero `wdbc.data`, obtenido del repositorio *UCI Machine Learning Repository* (<http://mllearn.ics.uci.edu/>). Este fichero corresponde al conjunto de datos *Breast Cancer Wisconsin*:

<http://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Diagnostic%29>

Aunque se ha dejado disponible el fichero que vamos a utilizar en el Campus Virtual de la asignatura, se recomienda a los alumnos que visiten estas páginas web para que puedan ver la información que contienen.

El fichero `wdbc.data` contiene datos clínicos de **569 mujeres con cáncer de mama**. Cada caso está descrito por **32 atributos o características**. El primero es un identificador y no es útil para la clasificación. El segundo contiene una letra, 'M' o 'B', que indica si el cáncer es maligno ('M') o benigno ('B'). Los restantes 30 atributos son diversos parámetros clínicos obtenidos para cada mujer.

Para poder utilizar este conjunto de datos, es necesario descargarse el fichero correspondiente (ya está disponible en el Campus Virtual) y sustituir los valores alfanuméricos por valores numéricos. En este caso, sustituiremos los valores del segundo atributo: 'M' por '1' y 'B' por '0'. Esta sustitución ya está hecha en el fichero que tenéis disponible en el Campus Virtual.

En el script que se cree, lo primero que tienen que hacer los alumnos es cargar los datos, eliminando la columna de identificador que no nos proporciona información de interés:

```
>> bcddata = csvread('wdbc.data', 0,1);  
>> size (bcddata)  
ans =  
    569     31
```

La tarea es tratar de **clasificar cada caso en un tipo de cáncer**. Por lo tanto, la primera tarea del alumno es organizar la matriz `bcddata` para que esté en la forma que vamos a necesitar para entrenar la red neuronal (cada caso o patrón ha de ser una columna, como

hemos visto en los ejemplos anteriores). A continuación, se han de separar los vectores de características y objetivos. El vector de objetivos (vamos a llamarlo `target`) nos dirá si el cáncer es maligno o benigno (segundo atributo en el fichero de datos, que era 'M' o 'B' en el fichero original y que ahora hemos sustituido por '1' y '0', respectivamente). La matriz de vectores de características (vamos a llamarla `indata`), contendrá los 569 casos, representados por 30 características clínicas cada uno.

Para obtener los rangos de variación de las entradas podemos emplear la siguiente función:

```
>> input_ranges = minmax(indata);
```

A continuación, se va a crear una red MLP con 10 nodos en la capa oculta:

```
>> net = feedforwardnet(10);
```

Y se van a ajustar algunos de sus parámetros para este problema:

```
>> net.inputs{1}.range = input_ranges;
```

```
>> net.layers{1}.transferFcn = 'tansig';
```

```
>> net.layers{2}.transferFcn = 'logsig';
```

```
>> net.trainFcn = 'trainlm';
```

Se puede observar que se ha elegido `trainlm` como algoritmo de entrenamiento (es el algoritmo de entrenamiento por defecto, no haría falta cambiarlo explícitamente) y se ha utilizado una red MLP con una sola capa oculta y con 10 neuronas en dicha capa. Esto es simplemente una configuración inicial que los alumnos pueden variar a lo largo de la resolución de este ejercicio. Además, se han ajustado las funciones de activación de las neuronas en las capas oculta y de salida y el rango de valores de entrada.

Como tenemos bastantes ejemplos en el conjunto de datos vamos a dividirlo en dos grupos: un conjunto de entrenamiento y un conjunto de test. Con el conjunto de entrenamiento se configurarán los parámetros de la red y se entrenará la misma. Una vez que esto se haya realizado, se empleará el conjunto de test para comprobar el funcionamiento de la red ante nuevas entradas (capacidad de generalización). Podemos separar las muestras al 50% de la siguiente forma, por ejemplo:

```
>> training_in = indata(:,1:2:length(indata));
```

```
>> training_target = target(1:2:length(target));
```

```
>> testset.P = indata(:,2:2:length(indata));
```

```
>> testset.T = target(2:2:length(target));
```

Ahora ya puede entrenar la red con el conjunto de entrenamiento y ver los resultados obtenidos sobre el conjunto de test. Para ello, el alumno debe:

- Entrenar la red neuronal empleando el comando `train` y los datos del conjunto de entrenamiento (`training_in` y `training_target`). Intente visualizar la evolución del algoritmo durante el entrenamiento empleando las herramientas que ofrece Matlab®.
- Obtener los resultados que da la red entrenada para los vectores de características del conjunto de test (`testset.P`) empleando el comando `sim`. Compararlos con los resultados que se deberían obtener de acuerdo con los valores objetivo correspondientes (`testset.T`).

- Calcular la **precisión** (porcentaje de casos del conjunto de test correctamente clasificados) y el **porcentaje de error** (porcentaje de casos del conjunto de test mal clasificados) que se obtiene con esta configuración sobre el conjunto de test.

¿Cómo de buenos son los resultados obtenidos? A la vista de los mismos, modifique diferentes parámetros de la red para tratar de minimizar el error (o maximizar la precisión) sobre el conjunto de test. Emplee gráficas y/o tablas para presentar los resultados obtenidos y comparar diferentes configuraciones. ¿Cuál es el mejor resultado que se ha obtenido? ¿Con qué configuración?

6. EVALUACIÓN

La práctica 3 de laboratorio se evaluará mediante un examen práctico final, junto con el resto de prácticas de laboratorio.

7. BIBLIOGRAFÍA

Bishop, C. M. (1995) *Neural Networks for pattern recognition*. Oxford University Press, Oxford.

Haykin, S. (1999) *Neural Networks: a comprehensive foundation*. Prentice Hall International, Upper Saddle River.

Ian H. Witten, Eibe Frank, Mark A. Hall, *Data Mining. Practical Machine Learning Tools and Techniques*, 3rd ed., Elsevier, 2011.

Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.